

Einführung

Ziel dieser Übung ist es, das *SIMT-Ausführungsmodell* (Single Instruction, Multiple Threads) einer GPU messbar und damit sichtbar zu machen. Dazu habe ich mit PyOpenCL¹ zwei Mikro-Benchmarks gebaut. Der erste nutzt einen rechenintensiven *Kernel* (Funktion, die auf der GPU läuft) und vermisst den Durchsatz in *GFLOP/s* (Anzahl Gleitkomma-Operationen pro Sekunde) über die Problemgröße sowie über den Grad der *Warp-Divergenz* (Threads im selben Warp nehmen verschiedene Verzweigungen). Der zweite ist ein speicherintensiver Streaming-Kernel und untersucht die effektive Bandbreite für drei Zugriffsmuster sowie das Verbergen von Latenzen über die *Occupancy* (Anteil der gleichzeitig aktiven Warps am Hardware-Maximum). Als Vergleichspunkt dient jeweils eine CPU-Referenz auf Basis von NumPy.

Die OpenCL-Kernels werden zur Laufzeit vom Treiber übersetzt² und auf der Hardware ausgeführt, während die gesamte Steuerung über Python erfolgt. Weil Numpy Matrixoperationen hardware-optimiert in C bzw. C++ ausführt, sind auch die Vergleichsbenchmarks aussagekräftig. Die hier genannten Messungen wurden auf einer AMD Ryzen AI 9 HX PRO 370 CPU mit einer AMD Radeon 890M Graphics On-Board-GPU ausgeführt. Die implementierten Scripts sollten allerdings geräteunabhängig mit `python -m gpubench all` ausgeführt werden können.

Die folgenden Erklärungen beschreiben die hardwareseitigen SIMT-Mechanismen und ordnen die gemessenen Werte ein. Obwohl viele Begrifflichkeiten in den AMD nahen Dokumentationen anders sind (bspw. Warps vs. Wavefronts), hält sich dieses Dokument an die in der Vorlesung genutzten NVIDIA Namenskonventionen.

Aufbau und Messmethodik

Ein `device`-Modul wählt einen OpenCL-Kontext (Organisationsstruktur um den Kernel) und liest zunächst die Geräteeigenschaften aus. Das für die Tests genutzte Gerät meldet 16 Compute Units, eine maximale Work-Group-Größe von 1024 und ein bevorzugtes Vielfaches der Work-Group-Größe von 64. Dieses Vielfache nutze ich als Stellvertreter für die Warp-Breite (bei NVIDIA typischerweise 32, auf AMD-GPUs 64). Die Zeitmessung erfolgt über OpenCL-Profilng-Events, die ausschließlich die Kernel-Laufzeit auf dem Gerät erfassen und damit den Host-Overhead ausklammern. Jede Messung verwendet zwei Aufwärmäufe und meldet anschließend den Median aus mehreren Wiederholungen, um Einschwing- und Ausreißereffekte zu dämpfen. Alle Ergebnisse werden als JSON abgelegt.

Aufgabe 1: SIMT und Warp-Divergenz

Im rechenintensiven Kernel bearbeitet jedes Work-Item genau ein Element und führt darauf eine feste Zahl von FMA-Schritten (Fused Multiply-Add: $x = x \cdot \text{COEF} + \text{BIAS}$) mit je zwei FLOPs je Schnritt aus. Abbildung 1 zeigt den Durchsatz über der Problemgröße n . Bei kleinem n (1024 Work-Items) liegt der Durchsatz mit 23,4 GFLOP/s niedrig, weil zu wenige Threads vorhanden sind, um die Recheneinheiten auszulasten und Startkosten zu amortisieren. Mit wachsendem n steigt der Durchsatz und sättigt bei etwa 3085,3 GFLOP/s (bei $n \approx 6,7 \cdot 10^7$). Auf einer GPU markiert dies den Übergang, ab dem alle Compute Units mit genügend Warps gefüllt sind, sodass sich SIMT lohnt.

¹Andreas Klöckner. *PyOpenCL*. 2024. URL: <https://documen.tician.de/pyopencl/>.

²Khronos Group. *The OpenCL Specification*. 2024. URL: <https://www.khronos.org/opencl/>.

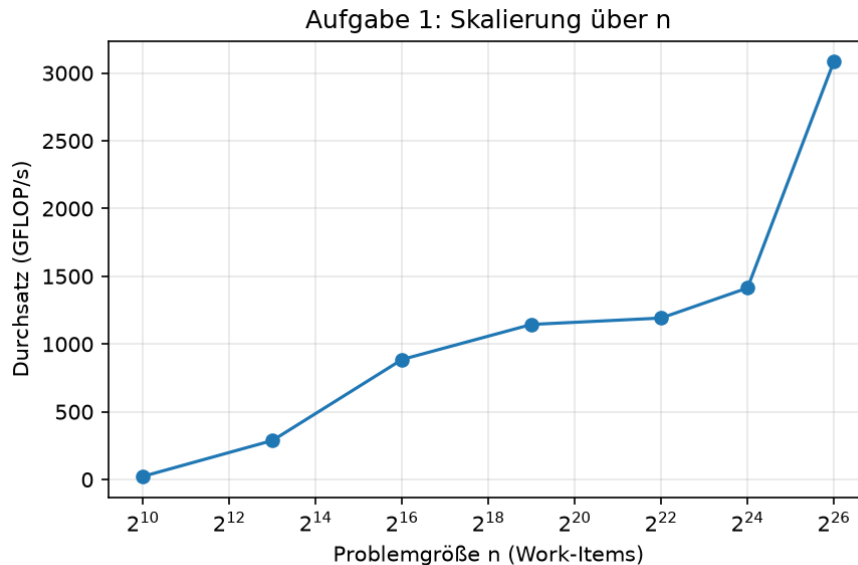


Abbildung 1: Durchsatz über der Problemgröße n (logarithmische n -Achse).

Den Kern der Aufgabe bildet die Warp-Divergenz. Im divergenten Kernel wählt jedes Work-Item anhand von `get_local_id(0) % DEGREE` einen von `DEGREE` Pfaden. Dabei leistet jeder Pfad exakt dieselbe Arbeit, sodass die Gesamtarbeit je Work-Item unabhängig vom Divergenzgrad konstant bleibt. Auf einer GPU führt ein Warp seine Lanes im Lockstep aus: Nehmen die Lanes unterschiedliche Verzweigungen, so arbeitet der Warp die `DEGREE` Pfade serialisiert nacheinander ab. In jedem Durchlauf ist nur die zu diesem Pfad gehörende Lane-Gruppe aktiv, während die übrigen Lanes maskiert mitlaufen und ihre Ergebnisse verworfen werden. Da jeder Durchlauf die volle Zeit kostet, fällt der Durchsatz annähernd auf $1/\text{DEGREE}$. Abbildung 2 zeigt die Messung und bestätigt die Theorie: Der Durchsatz sinkt von 3088,2 GFLOP/s (kein divergenter Pfad) auf 55,1 GFLOP/s bei Grad 64, also um den Faktor 56,1.

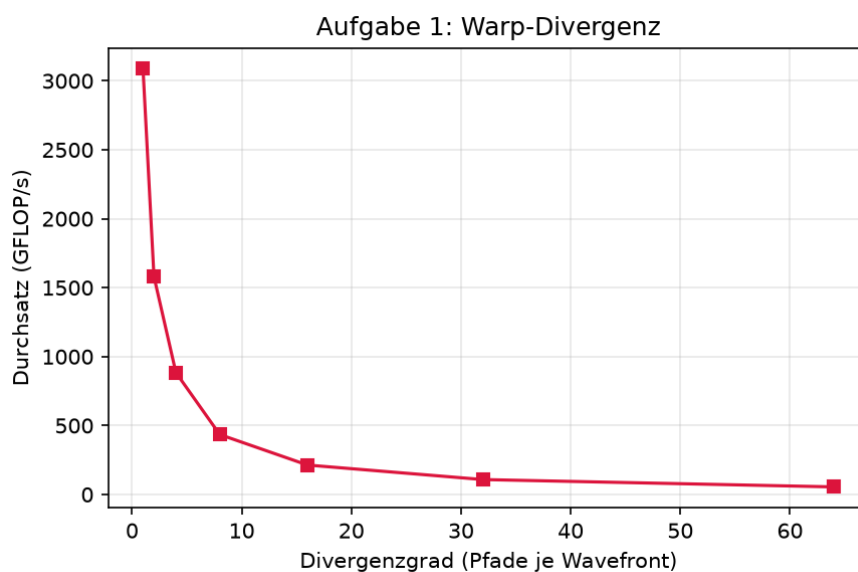


Abbildung 2: Durchsatz über dem Divergenzgrad (Pfade je Warp).

Dieser steile, annähernd mit $1/\text{DEGREE}$ verlaufende Einbruch ist die unmittelbare Folge der Lockstep-Ausführung, die divergente Pfade serialisiert. Die NumPy-Referenz auf der CPU bleibt im Gegensatz dazu über die Divergenzgrade nahezu konstant, da ein CPU-Kern Sprungvorhersage und unabhängige Pfadausführung besitzt und beliebig viele Verzweigungen damit kaum kosten.

Aufgabe 2: Speicherzugriff und Latency-Hiding

Der speicherintensive Kernel berechnet $b[i] = a[\text{idx}[i]] \cdot c$ und besitzt eine sehr geringe arithmetische Intensität, ist also bandbreitenbegrenzt. Das Indexfeld `idx` kodiert das Zugriffsmuster, sodass ein einziger Kernel alle drei Muster fair vermisst: coalesced (Identität), strided (großer, zu n teilerfremder Schritt) und gather (zufällige Permutation mit festem Seed). Abbildung 3 zeigt die effektive Bandbreite. Der coalesced-Zugriff erreicht 78,7 GB/s und liegt damit nahe an der separat gemessenen Kopier-Spitzenbandbreite von 79,4 GB/s. Der strided-Zugriff bricht auf 6,9 GB/s ein, der gather-Zugriff auf 6,9 GB/s; coalesced ist somit rund um den Faktor 11,3 schneller als gather. Auf einer GPU verschmilzt die Hardware benachbarte Zugriffe eines Warps zu wenigen Speichertransaktionen (Coalescing). Unregelmäßige Adressen zerfallen dagegen in viele einzelne Transaktionen und lasten die Bandbreite schlecht aus.

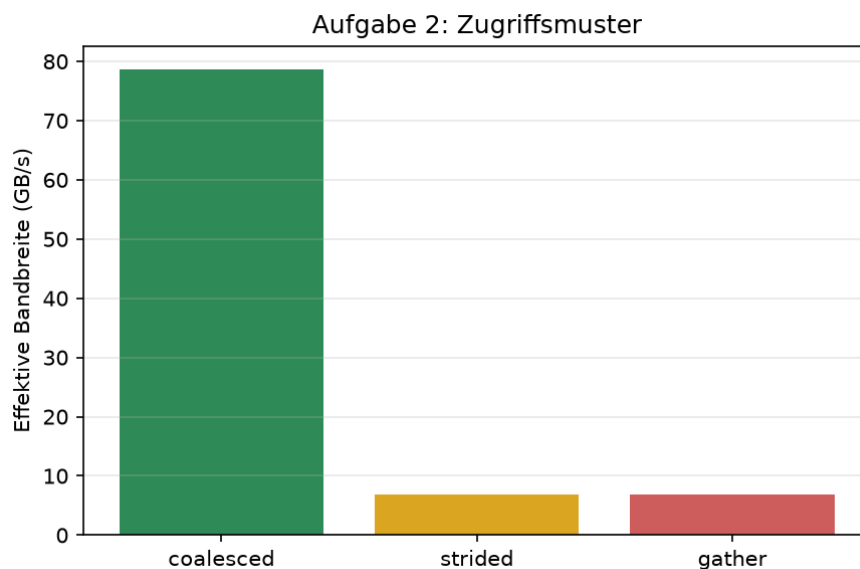


Abbildung 3: Effektive Bandbreite je Zugriffsmuster.

Abbildung 4 variiert die Work-Group-Größe und damit die Occupancy, also das Verhältnis aus aktiven zu maximal möglichen Warps je Compute Unit. Die Bandbreite steigt von 45,0 GB/s bei 8 Work-Items je Gruppe auf 80,2 GB/s bei 512 und läuft dann in ein Plateau. Die GPU verbirgt Speicherlatenz nicht über große Caches, sondern über massive Parallelität: Wartet ein Warp auf Daten, schaltet der Scheduler auf einen anderen lauffähigen Warp um. Erst wenn genügend Warps resident sind, wird die Latenz vollständig verdeckt. Die CPU verfolgt die entgegengesetzte Strategie und verbirgt Latenz über die Cache-Hierarchie und Prefetching: Die NumPy-Referenz erreicht sequenziell 12,9 GB/s, beim zufälligen gather aber nur 1,5 GB/s, weil dann nahezu jeder Zugriff einen Cache-Miss erzeugt.

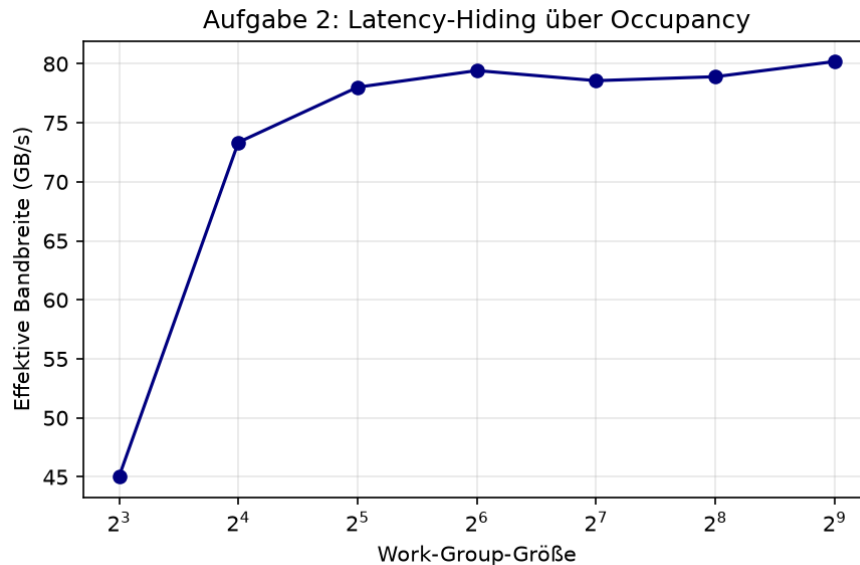


Abbildung 4: Bandbreite über der Work-Group-Größe (Occupancy).

Fazit

Tabelle 1 fasst die Messungen zusammen. Sie stellt die GPU AMD Radeon 890M Graphics (radeonsi, strix1, ACO, DRM 3.64, 7.0.12-1-cachyos) der NumPy-CPU-Referenz gegenüber.

Messung	GPU	NumPy (CPU)
Spitzen-Durchsatz (GFLOP/s)	3085,3	14,5
Divergenz-Einbruch (Grad 1 zu 64)	Faktor 56,1	nahezu konstant
Bandbreite coalesced (GB/s)	78,7	12,9
Bandbreite gather (GB/s)	6,9	1,5

Tabelle 1: Zusammenfassung der Messungen.

Beide Experimente zeigen dieselbe Konsequenz des SIMT-Modells: Hoher Durchsatz entsteht nur bei divergenzarmen Kontrollflüssen und bei regelmäßigen, coalesceten Datenlayouts. Wo eine CPU über Sprungvorhersage und Caches einzelne unregelmäßige Zugriffe abfedert, bestraft die GPU sie durch Serialisierung der Pfade und durch zerfallende Speichertransaktionen. Für GPU-Anwendungen folgt daraus unmittelbar, Datenstrukturen so anzuordnen, dass benachbarte Threads benachbarte Daten lesen, und Verzweigungen innerhalb eines Warps möglichst zu vermeiden.